

security group:

```
{{< engineering/stable-counterparts role="Software Supply Chain Security:Pipeline Security" >}}
```

Stable Counterparts

To find our stable counterparts look at the Pipeline Security product category listing.

JTBD

You can view and contribute to our current list of JTBD and job statements here.

Technologies

Like most GitLab backend teams we spend a lot of time working in Rails on the main GitLab app. Familiarity with Docker and Kubernetes is also useful on our team.

Common Links

- Issue Tracking Board
- Issue Backlog
- Slack Channel
- Group Direction Page
- GitLab Unfiltered YouTube Playlist

Our Repositories

- GitLab

How we work

Planning

General planning

Our team does not use Quad-Planning process because we do not have an assigned Software Engineer in Test. Instead we deploy a "tripod" approach where PM, EM, and UX work together to plan at least three milestones worth of work at a high level. During planning, the PM sets the general direction for the team and works with the EM to ensure the team's plans are ambitious, yet achievable based on the team's capacity. This ensures checks and balances during the planning discussions to minimize carryover and ensure engineering priorities are addressed along with feature priorities in the roadmap. UX will work with PM to ensure research and design issues are ready in time for implementation. Planning is mostly asynchronous via the release planning issue (see below), with additional discussion during weekly 1:1 sessions and team meetings as needed. This tripod approach allows us to address dependencies early on and ensure Design and Engineering are aligned on our product roadmap.

Release planning issue

We use a release planning issue to plan our release-level priorities over each milestone. This

issue is used to highlight deliverables, capacity, team member holidays, and more. This allows team members and managers to see at a high-level what we are planning on accomplishing for each release, and serves as a central location for collecting information. It is important to note this planning issue is static once the milestone starts and will not be updated to reflect changes to the milestone (e.g. a higher priority item is added to the milestone post-start and a planned issue is removed). This allows us to compare our baseline plan to final release as part of the retrospective issue.

Issue weighting and refinement

Before issues can be moved from the workflow::planning breakdown status into the workflow::ready for development status, they must have a weight greater than 0 applied to the issue.

We will apply a two-step estimating process:

- Phase 1: High-level/Rough estimate. Our intention here is to enable the PM to schedule issues according to capacity multiple milestones in advance, the EM will add a high-level estimated weight to issues. The PM will then schedule the issue to a specific milestone.
- Phase 2: Refinement/Validated estimate. Engineers will be assigned to refine and adjust the estimated weight (if necessary) during the first two weeks of the milestone prior to the issue's scheduled milestone.

The intention of this phased process is to enable agile planning and reduce the burden of our engineers in frequent context-switching required to investigate and refine issues throughout each milestone. Additionally, this provides our team a quarterly look ahead in the group's direction.

Issue weight system

An issue weight is determined based on the complexity in the following criteria:

- Code change - How difficult is it to make the required code changes?
- Interaction and dependency on other teams - How much interaction with other teams are needed?
- Path to production - What needs to be done to get the code changes safely in production (e.g feature flag, backward compatibility, etc)?
- Performance impact - What it takes to maintain high performance in the system?

Based on these criteria, an issue can have one of the following weights:

Weight	Description
--------	-------------

---	---
-----	-----

1: Trivial	Issues that are very well understood. The exact solution is already known and straight forward. The scope of change is very isolated. Examples are documentation updates, simple regressions, and bugs or technical debt that can be fixed with a few lines of code.
------------	---

2: Small	Issues that are well understood and have an outlined solution. No surprises are expected. No coordination with other teams or people is required. Examples are simple features, like a new API endpoint to expose existing data or functionality, or regular bugs or performance issues.
----------	---

| 3: Medium | Issues that are well understood and have an outlined solution. These issues require external team involvement or coordination to be released, such as a feature flag.

Examples are regular features, potentially with a backend and frontend component, or most bugs or performance issues. |

| 5: Large | Issues that are known to be complex. A solution has been outlined. There are many major edge cases that need to be catered for. Surprises are expected. Extensive coordination with other teams is required. Careful release process needs to be considered. These issues may have potential for adverse performance impact or catastrophic failures. They may also involve more one components (backend, frontend, gitaly, workhorse, runner, etc) or changes the interaction between the components.

Examples are issues that changes API contracts requiring backward compatibility, requires multiple feature flags to be safely released. It could also be issues where the team does not have any existing expertise or knowledge, or require changes in components that the team does not usually work on. |

| 8: Unknown | An issue that is weight 8 will not be scheduled and instead should be investigated further in order to be broken down into smaller issues

Examples are bugs that are not well understood or easily replicated, bugs or features that do not have a suggested solution. |

If the weight of an issue cannot be determined within a day, create a separate investigation issue for an in-depth investigation.

Design and Development collaboration

When issues require a design proposal, we follow the Product Development Flow. Design and development should work together from the start to ensure the issue follows our MVC guidelines, while still providing value and a usable experience.

To maintain a SSOT, the same issue should be used for design and development. This creates less duplicated work for both teams. Product designers should use the UX Definition of Done template to clearly state where the issue stands in the product development flow. An example of this in practice is <https://gitlab.com/gitlab-org/gitlab/-/issues/33418/>.

Once the design is complete, and appropriate workflow labels are applied, design, quality, and development (include FE, BE, and EM) should work together to break down the issue further for implementation, if necessary.

Technical Investigation

In the process of refinement we may discover a new feature will require a blueprint or the team feels input from maintainers will help scope down the problem, ensure the feature is performant and/or reduce future technical debt. When this happens the team will create a Technical Investigation issue for the investigation. This issue will be assigned to one team member. That team member should spend the minimum amount of time to create documentation, a Proof of Concept, or some other artifact that clarifies the approach to the problem, ideally in less than 5 working days. This will help us to gather information, validate the solution with others, and propose a plan to execute. They will answer specific questions outlined in the Technical Investigation issue before work on the feature is started. This process is analogous to the concept of a Spike.

When possible, the assigned team member is encouraged to schedule synchronous time with another

developer to pair on the investigation and publishing of the results (Example Technical Investigation issue [gitlab336617](#)). By default Technical Investigation issues are weighted at a 2 and we timebox them to 3 business days from start to presentation of data. Team members may change this weight and/or time frame at their discretion.

We limit the number of technical investigation issues assigned in each milestone to two so that we can maintain overall velocity and MR Rate.

Splitting issues

If an issue has several components (e.g. ~frontend, ~backend, or ~documentation) we should split it up into separate implementation issues. When these issues are created, the issues should be titled Frontend: [Issue title] or Backend: [Issue title], and marked as blocked by in case one is blocking the other. The original issue should hold all the discussion around the feature, with the implementation issues being used to track the work done. By splitting issues, there are several benefits:

1. There's only one DRI per issue.
1. Workflow labels and health statuses are more relevant.
1. We can weight issues with more accuracy.
1. We can mark one implementation as a blocker for another.
1. It's easier to see what work can be picked up by each functional group.
1. We can schedule feature work across multiple milestones.
1. When team members are selecting work from the group workflow board, it simplifies what issue relates to which component.

Error Budget issues

Error Budgets for stage groups have been established in order to help groups identify and prioritize issues that are impacting customers and infrastructure performance.

The Pipeline Security group Error Budget dashboard is used to identify issues that are contributing to the Pipeline Security group's error budget spend.

The engineering manager will review the error budget dashboard weekly to determine whether we're exceeding our budget, determine what (if anything) is contributing to our error budget spend, and create issues addressing root cause for product manager prioritization. Issues created to address error budget spend should be created using appropriate labels and prioritized according to our technical debt process.

Prioritizing technical debt and deferred UX

We follow the company guidance in how we prioritize technical debt and Deferred UX. In order to manage this effectively we have decided to keep track of technical debt, Deferred UX, and feature maintenance issues that are ready to be worked on in the "~workflow::scheduling" column, prioritized by Product, but informed by the impact these issues will have on our future velocity. We will try to dedicate a certain percentage (~30%) of our capacity by weight in each milestone to paying down technical debt and deferred UX. We do not consider bugs to be debt, and they are prioritized as part of the remaining capacity by weight.

Deliverables in a Milestone

Engineering managers apply the Deliverable label on issues that meet the following criteria:

- It is delivering value that is visible to the user
- It is near the top of the issue board (high priority)
- The issue is well-defined
- UX DoD is applied
- The issue is weighted
- The engineering manager believes we have the capacity to complete the work in the assigned milestone
- The team has accepted the issue into the milestone and moved it to ~"workflow::ready for development" status

Definition of Ready

Before the team will accept an issue into a milestone for work it must meet these criteria:

- Issues labeled with ~"type::feature" include a well stated "why" and customer problem
- Issues labeled ~"type::bug" include steps to reproduce
- Designs are in the design tab if needed
- If the issue will be worked on by Software Supply Chain Security:Pipeline Security engineers, it has a weight
- Design proposal satisfies the UX Definition of Done (DoD)

Definition of Blocked

Issues that depend on another issue to be completed before they can be validated on Canary are considered blocked, and should have the ~workflow::blocked label applied. The issues should also be marked as blocked within the related issues section of the issues.

Follow-up Issues

We utilize a process wherein issues that are follow-ups to other issues and are being worked on in the same milestone they are created will have a ~follow-up label added to them. Examples of items the team may create and label as ~follow-up include but is not limited to feature scope-creep, non-blocking requests from code review, additional UI polish, non-blocking refactoring that can be done, Low Priority (P2 or lower) bug fixes, etc.

Release post checklist issue

During each milestone, we create a Release Post Checklist issue that is used by team members to help track the progress of the team release posts. We use the checklist issue to associate release post merge requests with links to the implementation issue, links to updated documentation, and list which engineers are working on each issue. The checklist issue provides a single place to see all this information.

Workflow

Unless specifically mentioned below, the Software Supply Chain Security:Pipeline Security group

follows the standard engineering, product, and UX workflows.

Starting New Work

Software Supply Chain Security: Pipeline Security team members are encouraged to start looking for work starting Right to left on our milestone board. This is also known as "Pulling from the right". If there is an issue that a team member can help along on the board, they should do so instead of starting new work. This includes conducting code review on issues that the team member may not be assigned to if they feel that they can add value and help move the issue along the board.

Specifically this means, in order:

- Doing verification that code has made it to staging, canary, or production
- Conducting code reviews on issues that are workflow::in review
- Unblocking anyone in the workflow::in development column
- Then, finally, picking a Deliverable from the workflow::ready for development column.

The goal with this process is to reduce WIP. Reducing WIP forces us to "Start less, finish more", and it also reduces cycle time. Engineers should keep in mind that the DRI for a merge request is the author(s), just because we are putting emphasis on the importance of teamwork does not mean we should dilute the fact that having a DRI is encouraged by our values.

What do I do when there are no more Deliverable issues to take?

If there are no more Deliverable issues that a team member can "pull from the right", the team member can do one of the following:

- Take on an issue with the workflow::ready for development and SSCS::P2 labels.
- Refine issues with the workflow::planning breakdown and needs weight labels to help us prepare for upcoming milestones.
- Take the opportunity to work on a technical debt or low-hanging fruit issue that you are interested in. These issues often are not a high enough priority to be planned as deliverables, so engineers are encouraged to proactively tackle these issues when they have capacity.

Priority

We use a series of labels to indicate the highest priority issues in the milestone.

- The highest priorities for a given milestone would be issues labeled as Deliverable and SSCS::P1 issues.
- If there are Deliverable issues with the bug::vulnerability label as well, those should take top priority over other deliverables.
- We use the SSCS::P2 and SSCS::P3 labels to prioritize issues planned for future milestone deliverables.

If a team member believes a specific issue should be considered a Deliverable or a higher priority, they are encouraged to ping the product and engineering managers on the issues where we can discuss and decide. Note that issues need to be refined with a weight for them to be considered for the Deliverable label.

Code Review

Code reviews follow the standard process of using the reviewer roulette to choose a reviewer and a maintainer. The roulette is optional, so if a merge request contains changes that someone outside our group may not fully understand in depth, it is encouraged that a member of the Software Supply Chain Security:Pipeline Security team be chosen for the preliminary review to focus on correctly solving the problem. The intent is to leave this choice to the discretion of the engineer but raise the idea that fellow Software Supply Chain Security:Pipeline Security team members will sometimes be best able to understand the implications of the features we are implementing. The maintainer review will then be more focused on quality and code standards.

We also recommend that team members take some time to review each others merge requests even if they are not assigned to do so, as described in the GitLab code review process. It is not necessary to assign anyone except the initial domain reviewer to your Merge Request. This process augmentation is intended to encourage team members to review Merge Requests that they are not assigned to. As a new team, reviewing each others merge requests allows us to build familiarity with our product area, helps reduce the amount of investigation that needs to be done when implementing features and fixes, and increases our lottery factor. The more review we can do ourselves, the less work the maintainer will have to do to get the merge request into good shape.

This tactic also creates an environment to ask for early review on a WIP merge request where the solution might be better refined through collaboration and also allows us to share knowledge across the team.

UX Review

As part of the code review process, when an MR makes a user-facing change (no matter how small), it should be reviewed by a Product Designer. When in doubt, if the MR is related to an issue with the ~UX label, involve a Product Designer. Include screenshots or screen recordings of the changes in the MR description whenever possible to make the review process easier for everyone.

For iterations on features behind feature flags, even when the changes won't be user-facing right away, involve a Product Designer. If they feel that it would be more efficient to do so, the Product Designer may choose to defer their review of the feature-flagged feature until it is closer to being complete. This option gives them the flexibility to prioritize their workload as they see fit and avoid some of the noise generated by the MR review process.

Issue Health Status Definitions

- On Track - We are confident this issue will be completed and live for the current milestone. It is all downhill from here.
- Needs Attention - There are concerns, new complexity, or unanswered questions that if left unattended will result in the issue missing its targeted release. Collaboration needed to get back On Track within the week.
 - If you are moving an item into this status please mention individuals in the issue you believe can help out in order to unstick the item so that it can get back to an On Track status.
- At Risk - The issue in its current state will not make the planned release and immediate

action is needed to get it back to On Track today.

- If you are moving an item into this status please consider posting in the gpipeline-security channel in slack. Try to include anything that can be done to unstick the item so that it can get back to an On Track status in your message.

- Note: It is possible that there is nothing to be done that can get the item back on track in the current milestone. If that is the case please let your manager know as soon as you are aware of this.

Issue progress updates

When an engineer is actively working (workflow of ~workflow::"In dev" or further right on current milestone) on an issue they will periodically leave status updates as top-level comments in the issue. The status comment should include the updated health status, any blockers, notes on what was done, if review has started, and anything else the engineer feels is beneficial. If there are multiple people working on it also include whether this is a front end or back end update. An update for each of MR associated with the issue should be included in the update comment. Engineers should also update the health status of the issue at this time.

This update need not adhere to a particular format. Some ideas for formats:

text

Health status: (On track|Needs attention|At risk)

Notes: (Share what needs to be shared specially when the issue needs attention or is at risk)

text

Health status: (On track|Needs attention|At risk)

What's left to be done:

What's blocking: (probably empty when on track)

text

Update <date>

Health status: (On track|Needs attention|At risk)

What's left to be done:

MRs

1. !MyMR1

1. !MyMR2

1. !MyMR3

There are several benefits to this approach:

- Team members can better identify what they can do to help the issue move along the board
- Creates an opening for other engineers to engage and collaborate if they have ideas
- Leaving a status update is a good prompt to ask questions and start a discussion
- The wider GitLab community can more easily follow along with product development
- A history of the roadblocks the issue encountered is readily available in case of

retrospection

- Product and Engineering managers are more easily able to keep informed of the progress of work

Some notes/suggestions:

- We typically expect engineers to leave at least one status update per week, barring special circumstances
- Ideally status updates are made at a logical part of an engineers workflow, to minimize disruption
- It is not necessary that the updates happen at the same time/day each week
- Generally when there is a logical time to leave an update, that is the best time
- Engineers are encouraged to use these updates as a place to collect some technical notes and thoughts or "think out loud" as they work through an issue

Monitoring changes behind feature flags

In addition to the steps documented for developing with feature flags at GitLab Software Supply Chain Security:Pipeline Security engineers monitor their changes' impact on infrastructure using dashboards and logs where possible. Because feature flags allow engineers to have complete control over their code in production it also enables them to take ownership of monitoring the impact their changes have against production infrastructure. In order to monitor our changes we use this helpful selection of dashboards and specifically the Rails controller dashboard (Internal Only) for monitoring our changes in production. Metrics we evaluate include latency, throughput, CPU usage, memory usage, and database calls, depending on what our change's expected impact will be and any considerations called out in the issue.

The goal of this process is to reduce the time that a change could potentially have on production infrastructure to the smallest possible window. A side benefit of this process is to increase engineer familiarity with our monitoring tools, and develop more experience with predicting the outcomes of changes as they relate to infrastructure metrics.

Closing Issues

After an engineer has ensured that the Definition of Done is met for an issue, they are the ones responsible for closing it. The engineer responsible for verifying an issue is done is the engineer who is the DRI for that issue, or the DRI for the final merge request that completes the work on an issue.

Issues for community contributions

We want to create a welcoming environment for everyone who is interested in contributing to GitLab in anyway be it code, documentation, UX, or others.

If an issue is being resolved in a merge request that is made by a community contributor, the engineering manager will assign the associated issue to themselves to keep better track of it. The engineering manager will help to coach the community contributor through the merge request process, or be responsible for delegating that coaching to a team member engineer.

Labels

Category Labels

The Pipeline Security group supports the product marketing categories described below:

Label				
-----	-----	----	-----	---
Category:Secrets Management	Issues	MRs	Direction	Documentation
Category:Artifact Security	Issues	MRs	Direction	Documentation - TBD

Feature Labels

Label			Description
-----	-----	----	-----
CI job token	Issues	MRs	Relates to functionality surrounding CIJOBTOKEN available in the Build environment.
secrets storage	Issues	MRs	Relates to functionality surrounding the usage of secrets managers, including integration with secrets storage providers, in the Build environment.
external authentication	Issues	MRs	Relates to functionality surrounding tokens for external authentication available in the Build environment.
SLSA	Issues	MRs	Relates to functionality for Supply-chain Levels for Software Artifacts (SLSA).
ID tokens	Issues	MRs	Relates to functionality surrounding the usage of ID tokens (idtoken). ID tokens are JSON Web Tokens (JWTs) that can be added to a GitLab CI/CD job. They can be used for OIDC authentication with third-party services.

Meetings

Pipeline Security Group Weekly Meeting

This synchronous meeting is to discuss anything that is blocking, or notable from the past week. This meeting acts as a team touchpoint. We have two sessions of this meeting each week - one for APAC/EMEA timezones and another for AMER timezones. We record each meeting so that everyone can benefit from the discussions asynchronously.

Async Daily Standups

We use geekbot integrated with Slack for our daily async standup. The purpose of the daily standup meeting is to keep the team informed about what everyone is working on, and to surface blockers so we can eliminate them. The standup bot will run at 10am in the team members local time and ask 2 questions:

1. What will you do today?
1. Anything blocking your progress?

Async Monthly Retrospectives

We use a GitLab issue in this project for our monthly retrospective. The issue is created automatically towards the end of the current milestone. The purpose of the monthly retrospective issue is to reflect on the milestone and talk about what went well, what didn't go so well, and what we can do better.

Instead of waiting until the end of the milestone to add items to the retrospective issue, we encourage team members to add comments throughout the month. We have a slack reminder on our gpipeline-security channel to remind us to add items to the issue each Friday.

How to work with us

On issues

Issues worked on by the Pipeline Security group have a group label of ~"group::pipeline security". Issues that contribute to the SSCS stage of the DevOps toolchain have the ~"devops::software supply chain security" label.

You can tag a team member with @mention in the issue if you have someone specific to address. If you need to call the attention of the entire group, you can tag @gitlab-com/pipeline-security-group which notifies the entire team.

SECTION: engineering/development/sec/software-supply-chain-security/pipeline-security/jtbd.md

Overview

The goal of this page is to create, share and iterate on the Jobs to be Done (JTBD) and their corresponding job statements for the Testing categories within the Verify Stage. Our goal is to utilize the JTBD framework to better understand our buyers' and users' needs.

Goals

Utilize JTBD and job statements to:

- Understand our users' motivations.
- Validate identified use cases and solutions.
- Continuously test and iterate features to ensure we are meeting our customers' needs.
- Create a transparent view for our stakeholders into the current and future state of the product.

Jobs To Be Done

{{% product/jtbd "Pipeline Security" %}}

SECTION: engineering/development/sec/software-supply-chain-security/security-policies/_index.md

Mission

The Security Policies group's mission is to provide security and compliance teams with a way to enforce security controls globally in their organization. With tools developed by our team,

customers can prevent security risks by enforcing and automating security scans and requiring security approvals for proposed changes in their repository. For a comprehensive understanding of our vision and direction, we invite you to visit the Direction page at <https://about.gitlab.com/direction/securityriskmanagement/securitypolicies/securitypolicymangement/>.

Top Priorities FY25

Integrate Security Policies with other Software Supply Chain Security feature categories by allowing users to scope policies to selected Compliance Frameworks or Projects.

Allow for complex and customized Scan Execution Policies by introducing Pipeline Execution Action with the ability to enforce Custom CI YAML for selected projects and groups.

Help users understand why we require merge request approvals in their MRs by displaying Security Policy Violation details as a comment with helpful information.

Help customers with the slow rollout of features from the Security Policies team by customizing their policy configuration to allow them to select if they want to fail open or fail closed in case the policy or project is misconfigured.

Decrease performance impact on runners when enforcing scheduled Scan Execution Policies.
Ability to enforce policies on Organization Level.

Customer Outcomes we are driving for GitLab

1. Increase usage of Merge Request Approval Policies with detailed information explaining why we require approvals for given MR.
2. Speed up adoption time in enabling Security Policies by providing a way to enforce them gradually without blocking their current workflows for selected projects or projects with given Compliance Frameworks only.
3. Increase the flexibility of Scan Execution Policies and allow users limitless possibilities with enforcing any job with Pipeline Execution Action for selected projects and groups.
4. Help other groups introduce new types of policies to help customers automate and manage their security controls on group and organization levels.

Common Links

Slack channel: [gsrcmsecuritypolicies](#)

Slack alias: [@governsecuritypoliciesbe](#), [@governsecuritypoliciesfe](#)

Google groups: sec-govern-security-policies@gitlab.com

How we work

Prioritization

We use our Security Policies Priorities list in our direction page to track what we are doing, and what order to do it in. Each initiative includes these fields:

1. Name - Description and link to the epic
1. BE DRI / FE DRI - indicates the backend and frontend DRIs who will be actively involved.
1. Size
1. Target Release

Complete items are removed from the table once the code is in production without a feature

flag, and a release post, if applicable, has been merged. The epic is closed at this point.

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
```

```
  {{< tableau/filters "GROUPLABEL"="security policies" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="security policies" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="security policies" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="security policies" >}}
{{< /tableau >}}
```

Workflow

The Security Policies group largely follows GitLab's Product Development Flow.

Additional information about how we operate can be found on the Planning page.

Our current workflow is visualized as flowchart on the Workflow page.

Our current process on how we work on features is on the Feature process page

MR Reviews

We follow these guidelines when submitting MRs for review when the change is within the Security Policies domain:

1. The initial review should be performed by a member of the team, stage, or section. This helps the team by:

- Faster reviews, as the reviewer is already familiar with the domain

- Additional awareness of changes taking place within the domain

- Identifying changes that don't align with what is happening with the domain

1. For GraphQL changes, the MR should be reviewed by a frontend engineer as soon as possible. This helps to validate the interface, and allows changes to be made before tests are written.

Issue Boards

Security Policies Delivery Board

Primary board for engineers from which engineers can work. It's stripped down to only include the workflow labels we use when delivering software.

Security Policies Planning Board

Milestone-centric board primarily used by product management to gauge work in current and upcoming milestones.

Security Policies EM Board

Engineer-centric board used by engineering management to gauge how heavy a load engineer is carrying. Judged by the number of issues assigned to them.

Issue and Merge Requests labels

GitLab has a labeling convention for issues and Merge Requests. We follow this convention, though there are specific labels required to route artifacts to us. We use these labels to filter issues meant for us on our issue boards. They are also used for metrics and KPI reporting.

| Label | Meaning |

| ----- | ----- |

| ~"section::sec" | Identifies the issue or MR as belonging to the Sec Section's roadmap. |

| ~devops::govern | Identifies the issue or MR as belonging to the Software Supply Chain Security Stage's roadmap. |

| ~"group::security policies" | Identifies the Security Policies group as the collection of individuals who will work on the issue or MR. |

| ~"Category:Security Policy Management" | Identifies the issue or MR as being part of the Security Policy Management feature category. |

| ~backend | Identifies the issue or MR as being part of GitLab's backend. |

| ~frontend | Identifies the issue or MR as being part of GitLab's frontend. |

When creating a new issue, use the /copymetadata 373191 quick command to copy required labels.

Quality

How to classify MRs which need to run Package and QA?

It is advisable to manually trigger the Package and QA downstream E2E job in an MR and review the results when there are changes in:

GraphQL (API response, query parameters, schema etc)

Gemfile (version changes, adding/removing gems)

Vue files which contain "data-qa-selector" attribute because these are used in identifying UI elements in E2E tests

Database schema/query changes

Any frontend changes which directly impact vulnerability report page, MR security widget, pipeline security tab, security policies, configuration, license compliance page

To manually trigger the QA job:

1. Navigate to the pipelines tab of the MR.

1. Click the > arrow on the right of the Stages and click the package-and-qa item.

It's advisable to run the QA job on the latest pipeline at least once during the MR review cycle.

Monitoring

Stage Group dashboard on Grafana

Metrics the team tracks

Engineering productivity metrics

Total Open Bug by Severity

Overall MRs by Type %

Group MR Rate

Flaky Test Issues

Slow RSpec Tests Issues

Product metrics

Adoption of Security Policies features:

Number of projects with an assigned security policy project

Number of groups with an assigned security policy project

Number of open merge requests with at least one applicable scan result policy

Number of projects with at least one scan result policy

Number of users who have created merge requests with at least one applicable scan result policy

Number of users who have created merge requests in Projects that have an assigned security policy project

Unique count of Software Supply Chain Security actions in a given timeframe

Unique count of the Security Policies actions in a given timeframe

Unique count of the Security Policies visits in a given timeframe

Unique count of users creating merge requests with security policies in a given timeframe

Contributing

Local testing of licensed features

When a feature needs to check the current license tier, it's important to make sure this also works on GitLab.com.

To emulate this locally, follow these steps:

1. Export an environment variable: `export GITLABSIMULATESAAS=1[^1]`

1. Within the same shell session run `gdk restart`

1. Admin > Settings > General > "Account and limit", enable "Allow use of licensed EE features"

See the related handbook entry for more details.

Cross-stack collaboration

We encourage frontend engineers to contribute to the backend and vice versa. In such cases we should work closely with a domain expert from within our group and also keep the initial review internal.

This will help ensure that the changes follow best practice, are well tested, have no unintended side effects, and help the team be across any changes that go into the Security Policies codebase.

Community Contributions

The Security Policies group welcomes community contributions. Any community contribution should get prompt feedback from one of the Security Policies engineers. All engineers on the team are responsible for working with community contributions. If a team member does not have time to review a community contribution, please tag the Engineering Manager, so that they can assign the community contribution to another team member.

Footnotes

[^1]: There are many ways to pass an environment variable to your local GitLab instance. For example, you can create a env.runit file in the root of your GDK with the above snippet.

SECTION: engineering/development/sec/software-supply-chain-security/security-policies/feature_process.md

Process

Feature development is organized in an epic that collects feature related issues. Features are usually developed behind a feature flag. The goal of the feature development process is to default enable the feature flag.

Feature development epics are listed in team priorities

Epic's issues

- Title
 - Add 'BE' or 'FE' into the title to signify backend and frontend. The labels would signify this as well, but adding this to the title makes it easier to spot
- Milestone
 - The feature flag rollout issue's milestone is the milestone in which we plan to release the feature. Any issue above the feature flag rollout issue should have a milestone <= the feature flag rollout issue milestone
- Workflow label
 - Needs to be kept up to date so that users/PMs know the status of the epic and can plan their work appropriately (e.g. if all the issues have a label of ~workflow::refinement halfway through the milestone, then the epic probably won't be released that milestone).
- Order of issues in the "Child issues and epics" list
 - Any issue above the feature flag rollout issue is required for release
 - Any issue below the feature flag rollout issue is not required for release, but either a

nice to have or a backlog issue

Rolling out a feature epic

- All issues above the feature flag rollout issue should be closed before rolling out a feature flag globally on production
- The feature flag rollout issue should be the highest open issue on "Child issues and epics" to signify the state of the epic

Follow-up work

- When additional work is identified that is not required for a release of a feature, a separate follow-up/improvements epic should be created and the issues should be added to it
 - This additional epic should not go into the priorities list for the group as not all of the issues may be important enough to schedule for a specific milestone. Instead the issues should be prioritized individually
- When we close the feature flag rollout issue, which happens when the feature flag has been defaulted on and the feature flag cleanup issue has been created (per the rollout issue template), we close the feature epic and move any remaining issues to the follow-up/improvement epic

Bugs

- Bugs found before the feature flag is defaulted should be added to the epic and evaluated for impact
 - if a bug blocks the release, they should stay with the epic
 - if a bug does not block the release, they should be added to the follow-up epic
- Bugs found after the feature flag is defaulted on do not need to be added to any epic

SECTION: engineering/development/sec/software-supply-chain-security/security-policies/workflow.md

How do we prioritize our current work?

Please refer to the flowchart provided below to understand the process we follow when determining the next task to work on after completing all assigned tasks.

```
<div class="x-scrollable">  
<div style="width: 1800px;">
```

mermaid
flowchart TD

AA{Is there anything on the board
that is currently assigned to you
and you are not blocked?} -->|Yes| A{Take a look at assigned issues according to their states:}

AA -->|No| BA{Is there anything on
the board you can work on?}

A -->|workflow::verification| B(Verify the issue,
provide results as a comment
and close it.)

A -->|workflow::in review| C{All MRs for the issue are merged
, and the change is

already deployed?}

C -->|Yes| D(Verify the issue,
provide results as a comment
and unassign yourself.
Close it when you are not an author of the change,
otherwise once you are unassigned
the bot will select the next person to verify it.)

C -->|No| E(Wait for review and address comments
in your MRs if there are any.)

A -->|workflow::refinement| F{Are you reviewing
the implementation plan
or preparing it?}

F -->|reviewing| G(Review the plan, provide comments if needed,
unassign yourself and move to
workflow::ready for development when ready.)

F -->|refining| H(Provide implementation plan,
provide weight if it is a feature
and ask another team member to review.
Try to time-box this activity to 4h max.)

A -->|workflow::in dev / workflow::ready for development| I(Make sure it is in the workflow::in dev state.
Work on the issue until you have an MR ready,
then move it to workflow::in review)

BA -->|Yes| BB(Take a look at unassigned issues on the board
according to this order:
- workflow::verification,
- workflow::refinement,
- workflow::ready for development.

When you see another team member is assigned to the issue
and you would like to reduce their load and help them,
reach out to them asking how you can help.)

BA -->|No| BC(Go to Priorities page.)

BC --> BD{Are you a DRI for one of the Epics?}

BD -->|Yes| BE(Prepare implementation issues according
to the Epic Refinement Process.)

BD -->|No| BF(Take a look at the board
for future milestone for issues to in
workflow::refinement
or workflow::ready for development states
and start working on them.)

</div>

</div>

SECTION: engineering/devops/_index.md

Vision

Our goal is not merely to launch features, but to ensure they land successfully and provide real value to our customers. We strive to develop a best-in-class product that exceeds expectations across all user groups by meeting high-quality standards while ensuring reliability and maintaining an ease of operation and scalability to meet diverse customer needs. All team members should remain mindful of our target customers and the multiple platforms we support in everything we do.

Ensure our product excels in all aspects especially for our primary customer organization archetypes of large enterprises. This includes scalability, adaptability, and seamless upgrade paths. When designing and implementing features, always keep in mind compatibility for all our deployment options: self-managed, dedicated and Software as a Service (SaaS).

Develop our technical, diverse and global team to drive results that support our product and customer growth, while maintaining our values and unique way of working.

Mission

GitLab's unique way of working asynchronously, handbook first method, utilization of the product we develop and clear focus on our values enables very high productivity. We focus on constantly improving quality, usability and reliability of our product to reach maximum customer satisfaction. Community contributions and customer interactions rely on efficient and effective communication. We are a data-driven, customer experience first, open core organization delivering one secure, reliable, world leading DevSecOps platform. Join us in setting new standards, driving innovation, pushing the boundaries of DevSecOps, and consistently delivering exceptional results for our customers.

Fostering High-Performing Teams for Efficient Execution

In today's competitive landscape, GitLab's success hinges on our ability to build and maintain high-performing teams capable of efficient and rapid execution.

Key Focus Areas

1. Team Optimization

- Enhance communication and leadership within groups
- Implement efficient processes
- Develop situational awareness across teams
- Promote continuous upskilling towards group needs and/or the wider engineering division

2. Streamlined Delivery

- Identify and fix internal and external bottlenecks
- Focus on the swift delivery of results meeting acceptance criteria and general quality bar
- Look out for improvements in efficiency by enhancing the product and using all its capabilities

3. Quad Execution

- Clarify responsibilities within quads
- Encourage accountability amongst team members and counterparts
- Empower quads to execute independently with a clear vision

4. Innovation and Creativity

- Encourage research and ingenuity by defining clear paths for delivering promising ideas
- Tap into the underutilized creative potential within teams

We aim to create an environment where high-performing teams can thrive, innovate and execute efficiently, ultimately driving GitLab's competitive edge in the market.

Connect the Dots! In the Product and Between Teams

One of our product's greatest strengths and selling points is its unified platform approach. While we currently offer multiple functionalities within a single product, we should strive to further enhance this advantage by creating stronger connections between these different areas.

To fully leverage this strength, we need to:

1. Integrate features more seamlessly across the platform
2. Develop cross-functional capabilities that highlight the benefits of our unified approach
3. Ensure a consistent user experience throughout the product
4. Align and reuse foundational implementations across the whole platform

Additionally, it's crucial to improve collaboration at the team level especially to support this part of our mission.

We should:

1. Increase awareness of ongoing projects and initiatives across teams
2. Share knowledge about new technologies and innovations and how they are applied today
3. Promote understanding of functionalities in different product areas
4. Foster greater collaboration between teams throughout the entire engineering organization

Product with a Customer Focus

We will continue our strong partnership with Product to make GitLab the best, most complete DevSecOps platform on the planet. This coordination and prioritization requires a lot of work and effort to provide the right data and make the right decisions. We will focus on a variety of factors, but top of mind will be our parent department's direction to be customer focused.

Usability

Millions of customers use GitLab, so UX improvements can have a huge collective impact across all of these individuals. We support this effort both in the product development as well as in our architecture. This includes continued conversion of Pajamas components in order to continue to improve the performance experienced by users.

Development team members should also constantly suggest and investigate how to improve the overall user experience of the product. These can range from enhancing performance (actual and perceived), suggesting new technologies, solving user experience issues efficiently, etc.

Organization

plantuml
@startwbs
DevOps Engineering
Create
Code Review
Remote Development
Source Code
Import
Ops
Deploy
Package
Verify:Runner
Verify:CI Platform
Verify:Pipelines
Pipeline Authoring

Pipeline Execution
Plan
Product Planning
Project Management
Knowledge
@endwbs

The DevOps department is comprised of:

- Dev
- Ops

To better fulfill our goals and vision, we are following R&D interlock process.

Onboarding

Welcome to GitLab! We are excited for you to join us.
Here are some curated resources to get you started:

- Joining as an Engineer
- Joining as an Engineering Manager
- Core DevOps GitLab Project
- Core DevOps Google Calendar

Meetings

Meeting (Internal Only, limited access)	Frequency	DRI	Topics
----- ----- ----- -----			
SaaS Availability	Weekly		Infrastructure Incident corrective actions, Feature Change Lock status, Past Due Infradev, Error Budget, or Security updates
Core DevOps Meeting	Weekly	Michelle Gill	Standards discussions, questions, help needed, FYIs, process improvements, collaborating on engineering strategy, event planning, leadership communication dissemination, wider initiative brainstorming, OKRs
Product Quality Standup	Weekly	Michelle Gill	Coordination on shared quality objectives (bug burndown, instrumentation) across groups

Performance Indicators

To better measure our goals and vision, we are tracking Performance Indicators here.

People Processes

1. Promotions
 - Calibration preparation
1. Talent Assessments
1. Hiring Contractors
1. Recognition and Awards
1. Learning and Development
 - Mentoring at GitLab

- Security Awareness Training
- GitLab Performance Testing
- Getting started with the GraphQL API
- Database Engineering
- Reliability Training

How We Work

Incident Management

Team members in some job families contribute to incident management directly through an on-call schedule for Incident Managers.

Team members should complete onboarding so they can be added to the schedule when needed. These frequently asked questions cover exemptions and changing shifts.

- Incident Management process
- Incident Manager On Call onboarding

Daily Duties for Engineering Directors

The following is a non exhaustive list of daily duties for engineering directors, while some items are only applicable at certain time, though.

1. Review engineering metrics
 1. DevOps Performance Indicators
 1. Stage Performance Indicators
1. Review hiring dashboards
1. Personal todo list
1. Personal GitLab board(s) if any
1. Working groups that the director drives or participates in
 1. Action items in agenda documents
 1. Issue boards
 1. Slack channel
1. Infradev triage
 1. Follow up open questions and ensure appropriate handling of issues with regard to priority and severity
 1. Agenda document
 1. Infradev board
1. Performance refinement
 1. Follow up open questions and ensure appropriate handling of issues with regard to priority and severity
 1. Agenda document
 1. Performance board
1. Infrastructure Development Escalations
 1. Triage new issues, enhance Issue details and ensure appropriate handling based on priority and severity
 1. Sync discussions for infradev Issues are part of the GitLab SaaS Weekly Meeting
 1. Agenda document
 1. Infradev board
1. Holiday Emergency Contact Rotations

1. Review and approve security approvals for the GitLab project when required and informing the security engineering team when a security risk is accepted rather than being resolved prior to approval.

Cross-Functional Collaboration

Issues that impact code in another team's product stage should be approached collaboratively with the relevant Product, UX, and Engineering managers prior to work commencing, and reviewed by the engineers responsible for that stage.

We do this to ensure that the team responsible for that area of the code base is aware of the impact of any changes being made and can influence architecture, maintainability, and approach in a way that meets their stage's roadmap.

Architectural Collaboration

At times when cross-functional, or cross-departmental architectural collaboration is needed, the GitLab Architecture Evolution Workflow should be followed.

Follow the Sun Coverage

When cross-functional collaboration is required across global regions and time zones, it is recommended to adopt the Follow the Sun Coverage approach to ensure seamless global collaboration.

Decisions requiring approvals

At GitLab we value freedom and responsibility over rigidity. However, there are some technical decisions that will require approval before moving forward. Those scenarios are outlined in our required approvals section.

Security Vulnerability Handling

1. The development groups who introduce or consume the dependency of concern (e.g. gems, libs, base images, etc.) are responsible for resolving vulnerabilities detected against the dependency.
2. For business selected vendors that provide base images (RHEL's UBI8 for example), we need to wait for their patches, or need to log Deviation Request (DR) as viable resolutions. The VulnMapper, an automation developed by the Threat Management team, can create vendor dependency DRs to a large extent, but there are still cases that DR needs to be reported manually.
3. The assigned development group can redirect issues if the initial assignment was inaccurate, following the processes for shared responsibility issues and/or Shared responsibility functionality.

Ownership of Shared Services and Components

The GitLab application is built on top of many shared services and components, such as PostgreSQL database, Redis, Sidekiq, Prometheus and so on. These services are tightly woven into each feature's rails code base. Very often, there is need to identify the DRI when demand arises, be it feature request, incident escalation, technical debt, or bug fixes. Below is a

guide to help people quickly locate the best parties who may assist on the subject matter.

Ownership Models

There are a few available models to choose from so that the flexibility is maximized to streamline what works best for a specific shared service and component.

1. Centralized with Specific Team

1. A single group owns the backlog of a specific shared service including new feature requests, bug fixes, and technical debt. There may or may not be a counterpart Product Manager.

1. The single group is a specific team, meaning there is an engineering manager and all domain owner individuals reside in this team. The DRI is the engineering manager.

1. This single group is expected to collaborate closely and regularly in grooming and planning backlog.

1. This model may require consensus from the Product Management counterpart.

1. This model may fit a subject domain that experiences active development.

1. Centralized with Virtual Team

1. A single group owns the backlog of a specific shared service including new feature requests, bug fixes, and technical debt. There may or may not be a counterpart Product Manager.

1. The single group is a virtual team, meaning it consists of engineers from various engineering teams, for example maintainers or subject matter experts. Typically there isn't an engineering manager for this virtual team. The DRI is an appointed person in the group who may not necessarily be an engineering manager.

1. This single group is expected to collaborate closely and regularly in refining and planning backlog.

1. This model may fit a subject domain that's in maintenance mode.

1. Collectives

1. Collectives consist of individuals from existing teams who voluntarily rally around a shared interest or responsibility, but unlike Working Groups may exist in perpetuity. The shared interest could be a specific technology or system. Collective members feel a collective responsibility to weakly own, improve upon or otherwise steer the subject they govern.

1. This is a weaker form of the Virtual Team but introduces more structure than a fully decentralized model. It can be appropriate when some form of ownership is desirable where the subject has cross-cutting impact and wide reach and cannot clearly be allocated to any specific team.

1. Collectives do not have product or engineering managers, they are fully self-governed.

1. Members of the Collective sync regularly and keep each other informed about the shared interest. Problem areas are identified and formalized in the Collective, but are not logged into a Collective backlog. Instead a DRI is assigned who should put the task forward to the team with the greatest need for the problem to be resolved. This is to ensure that work is distributed fairly and that there are no two backlogs that compete with each other for priorities.

1. Collectives work best when they consist of a diverse set of individuals from different areas of product and engineering. They double as knowledge sharing hubs where information is exchanged from across teams in the Collective first, and then carried back by the individuals to their specific teams.

1. Decentralized

1. The team who implements specific functions or utilizes certain features of the shared services is responsible for their changes from local development environment to production deployment to continued maintenance post-deployment. There is not a development-wide single DRI

who owns a portion or the entirety of a shared service.

1. A specialty team may exist for specific subject domains, however their role is to enable scalability, availability, and performance by building a solid foundation and great tools for testing and troubleshooting for other engineering teams, while they are not responsible for gating every single change in the subject domain.

Shared Services and Components

The shared services and components below are extracted from the GitLab product documentation.

Service or Component	Ownership Model	DRI and Group (Centralized Only)	Additional Notes
Alertmanager	Centralized with Specific Team	@twk3	Distribution Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
Certmanager	Centralized with Specific Team	@twk3	Distribution Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
Consul			
Container Registry	Centralized with Specific Team		Package
Email - Inbound			
Email - Outbound			
Elasticsearch	Centralized with Specific Team	@changzhengliu	Global Search
GitLab K8S Agent	Centralized with Specific Team	@nicholasklick	Configure
GitLab Pages	Centralized with Specific Team	@vshushlin	Knowledge
GitLab Rails	Decentralized		DRI for each controller is determined by the feature category specified in the class. app/controllers and ee/app/controllers
GitLab Shell	Centralized with Specific Team	@andr3	Create:Source Code Reference
HAproxy	Centralized with Specific Team		Infrastructure
Jaeger	Centralized with Specific Team	@dawsmith	Infrastructure:Observability Observability team made the initial implementation/deployment.
LFS	Centralized with Specific Team	@andr3	Create:Source Code
Logrotate	Centralized with Specific Team	@plu8	Distribution Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
Mattermost	Centralized with Specific Team	@plu8	Distribution Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
MinIO	Decentralized		Some issues can be broken down into group-specific issues. Some issues may need more work identifying user or developer impact in order to find a DRI.
NGINX	Centralized with Specific Team	@plu8	Distribution
Object Storage	Centralized with Specific Team	@lmcandrew	Scalability::Frameworks
Patroni	General except Geo secondary clusters		Centralized with Specific Team @plu8
Patroni	Geo secondary standby clusters		Centralized with Specific Team @luciezhaogeo
PgBouncer	Centralized with Specific Team	@plu8	Distribution
PostgreSQL	Framework and Tooling		Centralized with Specific Team @alexives

/>Database | Specific to the development portion of PostgreSQL, such as the fundamental architecture, testing utilities, and other productivity tooling |
PostgreSQL GitLab Product Features	Decentralized		Examples like feature specific schema changes and/or performance tuning, etc.
Prometheus	Decentralized		Each group maintains their own metrics.
Puma	Centralized with Specific Team	@pjphillips Cloud Connector	
Redis	Decentralized		DRI is similar to Sidekiq which is determined by the feature category specified in the class. app/workers and ee/app/workers
Sentry	Decentralized		DRI is similar to GitLab Rails which is determined by the feature category specified in the class. app/controllers and ee/app/controllers
Sidekiq	Decentralized		DRI for each worker is determined by the feature category specified in the class. app/workers and ee/app/workers
Workhorse	Centralized with Specific Team	@andr3 Create:Source Code	

Supporting Customers

Working with Support

When DevOps collaborates with Support it provides invaluable insight into how customers are using the product and the challenges they run into. A few tips to make the process efficient:

- Get access to Zendesk so you view the question and communication from customers.
- Always write answers in a way that they can be "cut-and-pasted" and sent to a customer.
- Reference documentation in your responses and make updates to GitLab documentation when needed.
- Refer to existing issues and epics to reiterate our transparency value and to invite participation from the customer.
- If you are unclear about the support-development collaboration process or workflow then please refer to the handbook page how to use gitlab.com to request help from the GitLab development team

Customer Account Escalation coordination

If development is the DRI or actively participating in a Customer Account Escalation, consider the following:

- Be careful to not make commitments to customers without first talking to product management and development leaders to confirm the impact that commitment may have on other commitments.
- The customer will want to know when they can see the benefits of a change. They may not be familiar with GitLab practices for tracking and predicting due dates and milestones. Also, they may not be familiar with our workflows and associated labels nor the predictability of code review timelines, different timelines on releases to GitLab.com compared with releases for self-hosted customers and our use of feature flags.

markdown

Customers often don't rely on asynchronous communication at the level that GitLab does. Educate the customer on our practices and adapt to find a combined asynchronous and synchronous communication method and cadence that works for everyone.

Encourage customers to collaborate with us in epics, issues, and merge requests of interest. Keep in mind that they may not have access to ones that are confidential and/or may not be

comfortable or able to collaborate with us in this public forum.

Consider utilizing Google documents to collaborate with the customer as a backup for collaboration via epics, issues, and merge requests.

Consider utilizing a shared Slack channel to collaborate, adding the customers to our slack via "one Slack channel access requests". Example

In meetings, tell customers why we like to record them and ask if they are OK with doing so.

Consider using Chorus for scheduling the recordings to address legal requirements for recording meetings with customers.

In meetings, tell customers why we take notes before, during, and after the meeting, as it may not be natural for them to collaborate in this way.

Make sure the appropriate priority label is applied to all issues being tracked by the customer.

In the agenda for recurring meetings, track the items tracked by the customer in priority order at the top and review the status, next steps, customer DRI, and GitLab DRI for each.

Discuss in the meeting periodically.

Remind GitLab team members in Slack to update the status of items they are the DRI for before recurring meetings.

Post a link to the meeting notes and recording in a Slack channel for the customer escalation, so those who did not attend know that the notes and recording are available for review.

When there is an action item for someone in a meeting (whether they are present or not), tag them in an issue or MR (or in Slack) so they will see it.

Reducing the impact of far-reaching work

Because our te